

ResumeFetch

A resume retrieval feature for a job portal, built on an existing LinkedIn profile

Project Type	Business Analyst Portfolio Project
Domain	Recruitment / HR Tech
Role	Business Analyst
Duration	Self-initiated

Project Overview

Most job seekers already have an up-to-date resume sitting on their LinkedIn profile, yet almost every job portal still asks them to upload one from scratch for every application. ResumeFetch removes that step: a candidate pastes their LinkedIn profile URL, the system validates it and retrieves the resume already associated with that profile, and the candidate previews it, verifies it, and either downloads it or saves it to their job portal profile. I built this as a self-directed portfolio project to practice the full BA lifecycle on a feature that's narrow enough to reason about in detail but realistic enough to defend in an interview.

The Problem

Applying to multiple roles across different platforms means re-uploading a resume every single time, even when a current version already exists on the candidate's LinkedIn profile. Over time this creates a few consistent frustrations:

- Repeated manual upload of a resume the candidate has already put on LinkedIn
- Version drift, where the file on a given portal is older or less complete than the LinkedIn resume
- Drop-off during the application itself, since resume upload is a common point where candidates abandon a form

None of these are dramatic on their own, but together they add friction to a moment (the application) where platforms most want to reduce it.

Business Goal

The goal isn't to build a new product, it's to remove one specific piece of friction from an existing application flow. Success looks like candidates spending less time on the upload step, fewer abandoned applications at that step, and resumes on file that are more consistent with what candidates already maintain on LinkedIn.

Proposed Solution

ResumeFetch lets a candidate paste their LinkedIn profile URL instead of uploading a file. The system validates the URL, retrieves the resume already associated with that profile, and shows it in a preview screen alongside a Resume Information Verification section, where the parsed Work Experience and Skills are listed so the candidate can check that what's been retrieved is accurate. Once the candidate checks a consent box confirming

the information is accurate and up to date, they can download the resume as a PDF or save it directly to their job portal profile. If retrieval fails — because no resume is found, the URL is invalid, or the profile's visibility settings block access — the system shows an error screen where the candidate can try again or use Upload Resume Manually to add one manually, so the feature never blocks an application.

The flow, in order:

1. Candidate pastes their LinkedIn profile URL
2. System validates the URL
3. System retrieves the resume associated with that profile
4. Candidate previews the retrieved resume
5. Candidate reviews the parsed Work Experience and Skills information under Resume Information Verification
6. Candidate checks a consent box confirming the information is accurate and up to date
7. Candidate downloads the resume as a PDF, saves it to their job portal profile, or both
8. If retrieval fails, the candidate sees an error screen with the option to try again or use Upload Resume Manually

My Role as Business Analyst

I ran this end to end, the way I would on a real engagement:

- Defined the business problem and objectives
- Wrote the Business Requirements Document (BRD)
- Wrote user stories with acceptance criteria
- Mapped the user journey and edge cases (invalid URL, no resume found, retrieval failure)
- Built a UML use case diagram and a workflow diagram in Draw.io
- Designed low-fidelity wireframes in Figma
- Defined scope boundaries and success metrics

Functional Requirements

1. LinkedIn profile URL input and validation
2. Retrieval of the resume associated with the validated profile
3. Resume preview screen
4. Resume Information Verification section displaying parsed Work Experience and Skills, for verification only
5. Consent checkbox confirming the retrieved information is accurate and up to date
6. Download as PDF
7. Save to job portal profile
8. Error screen with Try Again and Upload Resume Manually options when retrieval fails
9. Error handling for invalid URLs, failed retrieval, and profile visibility settings that block access

User Journey

1. Candidate opens ResumeFetch
2. Candidate pastes their LinkedIn profile URL
3. Candidate submits the URL
4. System validates the URL
5. System retrieves the associated resume
6. Candidate previews the retrieved resume
7. Candidate views the parsed Work Experience and Skills information under Resume Information Verification
8. Candidate confirms the information is accurate and up to date
9. Candidate downloads the PDF or saves it to their profile
10. If retrieval fails, the system shows an error screen where the candidate can try again or use Upload Resume

Assumptions & Constraints

A few conditions this design assumes, worth stating plainly:

- The candidate provides a valid LinkedIn profile URL
- A resume is available for retrieval from that profile
- Internet connectivity is available during retrieval
- Upload Resume Manually is available as a fallback whenever retrieval fails
- Resume retrieval depends on the availability of the LinkedIn integration
- The candidate's LinkedIn profile visibility settings allow the resume to be accessed

Deliverables

- Business Requirements Document (BRD)
- Functional requirements
- User stories with acceptance criteria
- Workflow diagram
- UML use case diagram
- Low-fidelity wireframes
- This case study

Tools Used

- Figma — wireframes
- Draw.io — workflow and use case diagrams
- Google Docs — BRD and user stories

Key Business Value

The value case is straightforward: fewer manual steps at the point in the funnel where candidates are most likely to give up, resumes on file that better reflect what candidates already maintain on LinkedIn, and a small but real edge for the platform against competitors that still require a from-scratch upload every time.

Challenges I Ran Into

The main design question was how much the feature should do with the retrieved resume. Early on I considered letting candidates edit the retrieved information directly inside the portal, but that turns a narrow, focused feature into a document editor, with a very different set of requirements around versioning and formatting. I scoped that out and kept the feature to retrieval, preview, and verification: candidates can see the parsed Experience and Skills to confirm they're current, but the resume itself — the PDF — isn't editable within ResumeFetch. If something's out of date, the intended path is updating it on LinkedIn, not editing it inside this feature.

A related decision was making the consent checkbox a hard gate before download or save. It would have been simpler to skip straight to download once a resume was retrieved, but that risks candidates submitting a resume they haven't actually looked at. Requiring an explicit confirmation step keeps the accuracy check meaningful rather than a formality.

I also had to be deliberate about the failure path. Retrieval can fail for reasons outside the candidate's control — no resume on the profile, an invalid or mistyped URL, profile visibility settings that block access, or the LinkedIn integration being temporarily unavailable. Rather than treating that as an edge case to handle later, I built two recovery paths into the core workflow from the start: the candidate can try the retrieval again, or use Upload Resume Manually to add one manually, so a failed attempt never leaves them stuck mid-application.

Outcome

ResumeFetch is a small feature, deliberately so. What it's meant to demonstrate is how I work through a BA project end to end: starting from a real, everyday frustration, scoping the feature down to something a platform could realistically ship, and documenting it the way I would for a real stakeholder — BRD, user stories, diagrams, and wireframes that all agree with each other and with the workflow actually implemented.